МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Google Form Quiz Solver Bot

Требования

Янковский Э., Мелещенко С., Гордиенко Ю., Волнухина В.

Санкт-Петербург, 2026

Содержание

1 Введение **4**
1.1 Основные функции 4

2 Общие требования **5**
2.1 Управление версиями и сотрудничество 5
2.2 Целевые артефакты 5

3 Функциональные требования **6**
3.1 Аутентификация пользователей 6
3.2 Решение форм 6
3.2.1 Формат ввода 6
3.2.2 Обработка неподдерживаемых типов вопросов . . . 7
3.2.3 Формат ответа бота 8
3.3 История и управление формами 9
3.4 Статус и информация 11
3.5 Ограничения и лимиты 11
3.6 Источник данных для LLM 11
3.7 Сценарии использования (Use Cases) 12
3.7.1 Решение новой формы по ссылке 12
3.7.2 Повторное решение сохранённой формы (/rescore) . 12
3.7.3 Проверка статуса долгой операции (/status) 13

4 Общие требования **14**
4.1 Производительность 14
4.2 Надежность 14
4.2.1 Обработка некорректных ответов LLM 14
4.2.2 Смешанные формы (частичная поддержка) 14
4.3 Безопасность 14
4.4 Роли пользователей 15
4.5 Авторизация 15
4.6 Эндпоинты 15
4.6.1 Эндпоинт healthcheck 15
4.6.2 Эндпоинт auth 16
4.6.3 Эндпоинт users 16

5 Требования к технологическому стеку **17**

6 Требования к документации **18**

7 Руководство по сдаче проекта **19**

7.1 Требования 19

7.2 Архитектура 19

7.3 Полный проект 19

8 Глоссарий **20**

1 Введение

1.1 Основные функции

1. **Решение Google-форм**
   - Приём ссылки на форму через команду `/solve` или текстовое сообщение.
   - Извлечение ссылки с помощью регулярного выражения.
   - Генерация ответов через GigaChat (3 попытки при ошибках, таймаут 30 секунд).
   - Вывод списка вопросов с ответами и уровнями уверенности.
2. **История и управление формами**
   - Просмотр истории решений (`/history [day|week|month|all]`).
   - Автоматическое сохранение успешно решённых форм.
   - Просмотр сохранённых форм (`/myforms`).
   - Удаление сохранённой формы (`/remove_form <id>`).
   - Повторное решение (`/rescore <id>`).
3. **Статус запросов**
   - Проверка состояния обработки запроса (`/status <requestId>`) с возможными статусами: *Ожидает*, *В обработке*, *Завершен*, *Неудача*.
4. **Администрирование**
   - Для роли `ADMIN`: просмотр пользователей – endpoint (`/users`) с API-ключем.
   - Получение API-ключа через эндпоинт `/auth` с login и pass для доступа к HTTP-эндпоинтам.
5. **Справочная информация**
   - Команда `/help` выводит список всех доступных команд.

2 Общие требования

2.1 Управление версиями и сотрудничество

- Весь код должен быть размещен в репозитории GitHub.

- Репозиторий должен содержать файл ‘README.md‘ с подробным руководством по:

    - Настройке проекта локально.

    - Сборке и запуску приложения.

    - Развертыванию приложения с использованием Docker.

    - Взаимодействию с Telegram-ботом.

- Docker-образ приложения должен быть размещён на Docker Hub.

2.2 Целевые артефакты

- Приложение должно создавать исполняемый "fat JAR" как основной артефакт.

- Приложение должно быть развертываемо как Docker-контейнер.

- Использовать инструмент сборки Gradle для управления зависимостями, сборки и упаковки приложения.

3 Функциональные требования

3.1 Аутентификация пользователей

Пользователи идентифицируются по id чата в Telegram, авторизация не требуется.

3.2 Решение форм

1. Получение адреса формы от пользователя:

   - пользователь может написать `/solve short_url`,
   - пользователь может написать `/solve long_url`,
   - пользователь может написать текстовое сообщение содержащее url адрес формы;

   в каждом из этих случаев бот забирает ссылку в обработку и сообщает об этом пользователю.

2. После проверки формы на соответствие требованиям через запрос к Google Forms API, бот сообщает пользователю о результате.

3. Получение ответа от LLM:

   - если LLM не ответила за 30 секунд, то есть ее время ответа привысило лимит, пользователь получает от бота сообщение об этом с указанием номера попытки обращения и общего количества попыток;
   - если LLM ответила пользователь получает сформированный ответ в виде списка из вопросов с ответами. В случае, если в вопросе были варианты ответа, рядом с ними указываются уровни уверенности модели в ответах.

3.2.1 Формат ввода

Бот принимает команды, начинающиеся с /, а также текстовые сообщения, если они содержат ссылку на форму, остальная часть сообщения игнорируется.

В случае, если после / идет неизвестная команада, пользователю сообщается об этом.

Проверка корректности ссылки, еще до запроса к Google Forms API, а также поиск ссылки в текстовом сообщении выполняется с использованием регулярного выражения, приведенного в листинге 1.

Листинг 1. Регулярное выражение задающее ссылки на формы Google Forms

```
^(https?:\/\/)?(www\.)?(docs\.google\.com\/(u\/\d\/)?forms\/d\/(e\/)?[a-zA-Z0-9_-]+(\/viewform)?|forms\.gle\/[a-zA-Z0-9_-]+)(\?.*)?$
```

3.2.2 Обработка неподдерживаемых типов вопросов

Система поддерживает автоматическое решение только определённых типов вопросов Google Forms. Остальные типы вопросов пропускаются – по ним не генерируется ответ, а пользователю выводится соответствующая пометка.

Поддерживаемые типы вопросов

Для следующих типов вопросов система должна сгенерировать ответ с помощью LLM:

- TEXT - краткий текстовый ответ (одна строка). LLM генерирует текстовый ответ (строка).
- PARAGRAPH_TEXT - развёрнутый текстовый ответ (абзац). LLM генерирует развёрнутый текст.
- MULTIPLE_CHOICE - радиокнопки, можно выбрать только один вариант из списка. LLM выбирает один вариант из предложенных, выводится уверенность в выбранном варианте.
- CHECKBOXES - флажки (несколько вариантов). LLM выбирает несколько вариантов, выводится уверенность по каждому выбранному варианту.
- DROP_DOWN - раскрывающийся список. LLM выбирает один вариант из выпадающего списка, аналогично MULTIPLE_CHOICE.
- LINEAR_SCALE - числовой диапазон. LLM выбирает число в заданном диапазоне, выводится уверенность.

- DATE - тип «Дата». LLM генерирует дату в формате ГГГГ-ММ-ДД.
- TIME - тип «Время». LLM генерирует время в формате ЧЧ:ММ.

Неподдерживаемые типы вопросов

Для следующих типов вопросов система не генерирует ответ. Вместо этого вопрос помечается как пропущенный с указанием причины:

- FILE_UPLOAD - загрузка файла. Пропускается с пометкой «загрузка файла не поддерживается».
- GRID - сложный тип «Сетка». Пропускается с пометкой «тип вопроса "сетка"не поддерживается».
- CHECKBOX_GRID - сетка с флажками. Пропускается с пометкой «тип вопроса "сетка с флажками"не поддерживается».
- IMAGE - вопрос, основанный на изображении. Пропускается с пометкой «вопросы на основе изображения не поддерживаются».
- VIDEO - вопрос, основанный на видео. Пропускается с пометкой «вопросы на основе видео не поддерживаются».
- При вопросах подразумевающих ввод личных данных (ФИО, email, дата рождения, текущее время, телефона и т.п.) бот отмечает этот вопрос в ответе как пропущенный, по причине «вопрос с личными данными».
- Прочие вопросы, не входящие в «Поддерживаемые типы вопросов». Пропускается с пометкой «неподдерживаемый тип вопроса».

3.2.3 Формат ответа бота

1. Успешное решение формы.

Бот отправляет сообщение в Telegram. Сообщение должно содержать:

- заголовок – название формы (берётся из Google Forms API) или, если недоступно, «Форма»;
- список вопросов – каждый вопрос начинается с новой строки и включает:

– номер вопроса;

– текст вопроса с ограничением в 250 символов, при превышении длины, конец обрезается и добавлеяется "...";

– сгенерированный ответ;

– для вопросов с уровнем уверенности – уровень уверенности модели в процентах (от 0 до 100%).

2. Частичное решение (часть вопросов не поддерживается или пропущена).

Бот обязан сообщить, что форма решена не полностью. В сообщение добавляется строка с предупреждением в начале о частичном решении (некоторые вопросы не поддерживаются), для каждого пропущенного вопроса вместо ответа указывается причина.

3. Полное отсутствие поддерживаемых вопросов (форма не может быть решена).

Бот отправляет сообщение с отказом и объяснением причины.

4. Ошибки при решении (таймаут LLM, недоступность API, неверная ссылка).

Все ошибки должны включать: краткое описание ошибки и информацию о том, что делать пользователю (повторить позже, проверить ссылку и т.п.).

5. Формат ответа на команду /status.

Должен содержать:

- requestId;
- текущий статус (Ожидает, В обработке, Завершен, Неудача).

3.3 История и управление формами

Пользователь имеет возможность:

1. Просматривать историю решённых форм.
   - Команда `/history [период]` выводит список всех попыток решения форм пользователем. Сюда попадают как успешные, так и неудачные запросы.

- Параметр `период` может принимать значения: `day` (за день), `week` (за неделю), `month` (за месяц) или `all` (вся история). Команда без параметра `период` выводит список за неделю (параметр по умолчанию week).
- Для каждого запроса указываются: дата, название формы, статус (`Ожидает`, `В обработке`, `Завершен`, `Неудача`) и `requestId`.

2. Сохранять формы для быстрого доступа.

   - После успешного решения форма автоматически сохраняется в личном списке пользователя с уникальным `id_формы`.
   - Сохранённую форму можно вызвать позже командами `/get_form <id_формы>`.

3. Удалять сохранённые формы.

   - Команда `/remove_form <id_формы>` удаляет форму из личного списка (история решений при этом не удаляется).

4. Просматривать список сохранённых форм.

   - Команда `/myforms` выводит список уникальных форм, которые пользователь когда-либо успешно решил (хотя бы один вопрос отвечен).

5. Просмотр ответов сохранённой формы.

   - Команда `/get_form <formId>` выводит последние сгенерированные ответы для формы с указанным идентификатором. Если форма никогда не решалась успешно – возвращается ошибка «Форма не найдена или нет успешных решений».

6. Повторно решать форму.

   - Команда `/rescore <id_формы>` запускает повторную генерацию ответов. Система создаёт новую запись в истории. При этом обновляется дата последнего решения для данной формы в выводе /myforms.

3.4 Статус и информация

- Запрос статуса обработки: команда `/status <requestId>` возвращает текущий статус запроса:
  - `Ожидает` — запрос принят, ожидает обработки.
  - `В обработке` — идёт генерация ответов LLM.
  - `Завершен` — форма успешно решена, ответы готовы.
  - `Неудача` — произошла ошибка.
- Справка: команда `/help` выводит список всех доступных команд с кратким описанием.

3.5 Ограничения и лимиты

- Активные запросы: один пользователь не может иметь более 1 запроса в статусе `Ожидает` или `В обработке` одновременно.
- Объём хранимых данных: на одного пользователя выделяется не более 10 МБ.
- Общее количество пользователей: система рассчитана на обслуживание не более 1000 активных пользователей.
- Требования к форме (не обрабатываются):

  - Формы, требующие авторизации Google (не публичные).

  - Формы, не содержащие ни одного поддерживаемого типа вопроса.
- Максимальное количество вопросов в форме: 50.
- Максимальная длина текста вопроса: 500 символов. Вопросы длиннее обрезаются до 500 символов с добавлением «...» в конце.

3.6 Источник данных для LLM

- Бот использует API стороннего LLM-провайдера — GigaChat.
- При недоступности провайдера возвращается ошибка.

3.7 Сценарии использования (Use Cases)

3.7.1 Решение новой формы по ссылке

Действия пользователя:

1. Пользователь отправляет боту ссылку на Google Forms (через команду /solve.

Реакция системы:

2. Бот подтверждает принятие ссылки в обработку.

3. Бот проверяет корректность ссылки (регулярное выражение) и доступность формы через Google Forms API. Отправляет пользоватлю результат проверки: если не проходит проверка ссылки бот просит отправить ссылку снова; если форма недоступна бот сообщает об этом.

4. Бот отправляет запрос к LLM (GigaChat) с таймаутом 30 секунд, при ошибках – до 3 попыток. При каждой повторной попытке бот сообщает пользователю о неудачной попытке с номером попытки и общем макисмальным количеством попыток.

5. Бот формирует ответ согласно разделу 3.2.3 и отправляет пользователю.

6. Если решение успешно (хотя бы один вопрос отвечен), форма автоматически сохраняется в личном списке пользователя (/myforms) и добавляется в историю (/history).

Пользователь получает сообщение с ответами на вопросы формы (или частичный ответ с пометками о пропущенных вопросах).

3.7.2 Повторное решение сохранённой формы (/rescore)

Действия пользователя:

1. Пользователь вводит команду /rescore <id_формы>, где id_формы – идентификатор из списка /myforms.

Реакция системы:

2. Бот находит сохранённую ссылку на форму.

3. Бот выполняет те же шаги, что и в решении новой формы по ссылке (проверка доступности, запрос к LLM, генерация ответа).

4. Создаётся новая запись в истории решений (/history).

Пользователь получает свежие ответы по форме, история пополня-

ется новой записью.

3.7.3 Проверка статуса долгой операции (/status)

Действия пользователя:

1. Пользователь отправляет команду /status <requestId>, где requestId был получен ранее (например, при отправке формы).

Реакция системы:

2. Бот ищет запрос по requestId в базе данных.

3. Бот возвращает сообщение, содержащее:

- requestId;

- Текущий статус: Ожидает, В обработке, Завершен, Неудача.

Пользователь узнаёт текущее состояние обработки своего запроса.

4 Общие требования

4.1 Производительность

1. Максимальное время ответа бота – 300мс, за исключением задержек внешних API.

2. Количество единовременных подключений пользователей до 1000.

4.2 Надежность

Повторные попытки при ошибках доступа к LLM. 3 попытки с интервалом в 30 секунд, если ответ не поступил, или 1 секунда, если была получена ошибка.

4.2.1 Обработка некорректных ответов LLM

Если LLM вернул ответ, который не соответствует структуре вопроса (например, для вопроса с вариантами ответа вернул текст, не совпадающий ни с одним вариантом, или для CHECKBOXES вернул не массив), система повторяет запрос к LLM один раз с уточняющим промптом, явно указывающим ожидаемый формат. Если повторный запрос также не дал корректного ответа – вопрос помечается как «ошибка генерации». Остальные вопросы решаются в обычном режиме.

4.2.2 Смешанные формы (частичная поддержка)

Если форма содержит как поддерживаемые, так и неподдерживаемые типы вопросов, система решает поддерживаемые, а неподдерживаемые пропускает с пометкой (см. раздел 3.2.2). В ответе обязательно указывается, что решение частичное.

4.3 Безопасность

1. Данные подключения к базе данных, а также необходимые ключи для внешних API хранятся в переменных окружения.

2. Ввод пользователя валидируется. По списку команд или, в случае с текстовым сообщением, регулярным выражением на наличие ссылки на форму.

4.4 Роли пользователей

1. Пользователь (User) – любой пользователь, идентифицируется по chat_id:
   - Стандартная роль для любого клиента,
   - стандартные права доступа,
   - отправляет запросы через Telegram бота.
2. Администратор (Admin) – пользователь, который может идентифицировать себе по API ключу:
   - API ключ можно получить на endpoint-е `/auth`, передав параметры pass и login.
   - По ключу API может получать доступ к endpoint-у `/users`.

4.5 Авторизация

1. Поддержка авторизации через Telegram по Chat ID.
2. Поддержка авторизации через HTTP для API:
   - ключи доступа создаются при запросе на `/auth`, при передаче правильных входных данных, которые хранятся в зашифрованном виде в БД; ключи хранятся 15 минут после создания.
   - предназначена только для пользователей группы Admin.

4.6 Эндпоинты

4.6.1 Эндпоинт healthcheck

- Не требует авторизации по ключу API.
- Должен возвращать статус сервера и список авторов проекта.

4.6.2 Эндпоинт auth

- Необходим для получения ключа API.
- Должен проинимать 2 параметра: login и pass для проверки администратора.
- Должен возвращать строку – ключ API, действительный в течении 15 минут.

4.6.3 Эндпоинт users

- Должен быть доступен только по ключу API, то есть для пользователей группы администраторов.
- Должен предоставлять список всех пользователей.

5 Требования к технологическому стеку

Язык программирования: Java 25.

Фреймворк: Spring 7 (WebFlux, Data MongoDB Reactive, Modulith).

Telegram API: Приложение должно интегрироваться с Telegram API для предоставления функциональности бота.

Брокер сообщений: Kafka.

База данных: MongoDB.

Контейнеризация: Docker, Docker Compose.

Тестирование: JUnit.

6 Требования к документации

Должна быть написана документация для API с использованием Spring Docs.

7 Руководство по сдаче проекта

7.1 Требования

Предоставить документ с требованиями, который включает:

- Цель и функциональность приложения.

- Ключевые функции Telegram-бота.

- Нефункциональные требования.

- Любые предположения или ограничения.

7.2 Архитектура

Предоставить документ по архитектуре, который включает:

- Высокоуровневый обзор дизайна системы.

- Диаграммы (например, компонентные диаграммы, диаграммы последовательностей или блок-схемы) для иллюстрации архитектуры.

- Обоснование выбранного технологического стека.

- Описание того, как приложение будет строиться, развертываться и запускаться.

7.3 Полный проект

Предоставить полный проект в виде:

- Ссылки на репозиторий GitHub с полной кодовой базой.

- Ссылки на Docker Hub, где опубликован Docker-образ.

- Ссылки или имя пользователя Telegram-бота для тестирования и взаимодействия.

8 Глоссарий

Термин	**Определение**
Telegram	Кроссплатформенный мессенджер с поддержкой облачного хранения сообщений.
Telegram-бот	Автоматизированная программа, работающая внутри Telegram и обрабатывающая команды пользователей.
Chat ID	Уникальный числовой идентификатор чата, группы, канала или диалога с пользователем в Telegram, используемый для аутентификации.
Git	Распределённая система контроля версий для отслеживания изменений в файлах и координации работы над проектом.
GitHub	Платформа для хостинга и совместной разработки проектов с использованием Git.
Репозиторий GitHub	Хранилище кода, где фиксируется весь код проекта и история изменений.
README.md	Файл с инструкциями по настройке, сборке, запуску и развертыванию приложения.
Fat JAR	Исполняемый файл Java, включающий все необходимые зависимости для запуска приложения без дополнительных библиотек.
Docker	Платформа для разработки, доставки и запуска приложений в контейнерах.
Docker-контейнер	Изолированный пакет, содержащий приложение и все его зависимости.
Docker Compose	Инструмент для определения и запуска многоконтейнерных Docker-приложений с помощью YAML-файлов.
Docker Hub	Облачный реестр для хранения и распространения Docker-контейнеров.

Продолжение на следующей странице

Термин	**Определение**
YAML-файл	Текстовый файл в формате YAML, используемый для описания конфигураций.
БД (база данных)	Структурированная система хранения данных для эффективной организации и извлечения информации.
API	Набор инструментов для взаимодействия между программами, позволяющий обмениваться данными и функциональностью.
API-ключ	Уникальный идентификатор для аутентификации и авторизации при доступе к API.
Telegram API	Интерфейс для создания Telegram-ботов и интеграции функций Telegram в приложения.
Фреймворк	Комплекс готовых решений и инструментов, ускоряющих разработку программного обеспечения.
Spring Framework	Платформа для разработки Java-приложений, предоставляющая инструменты для создания веб-приложений и микросервисов.
Reactive Spring (WebFlux)	Модуль Spring Framework для создания реактивных, неблокирующих веб-приложений.
MongoDB	NoSQL база данных, хранящая данные в формате документов.
Брокер сообщений	Система, управляющая обменом сообщений между приложениями в распределённой архитектуре.
Kafka	Распределённый программный брокер сообщений.
Spring JPA	Инструмент Spring Framework для упрощения взаимодействия с реляционными базами данных.

Продолжение на следующей странице

Термин	**Определение**
ORM	Технология преобразования данных между несовместимыми типами систем (объектно-ориентированные языки и реляционные БД).
Spring Modulith	Инструмент для структурирования монолитных приложений в модули.
HTTP	Протокол передачи данных в интернете.
Токен	Уникальная строка данных для аутентификации и авторизации.
Эндпоинт (endpoint)	URL-адрес или точка доступа в веб-сервисе для взаимодействия клиента с сервером.
Healthcheck	Специальный эндпоинт для проверки состояния и работоспособности сервера или приложения.
Логирование	Процесс записи событий и действий в системе для мониторинга и анализа.
Валидация	Проверка данных на соответствие заданным критериям.
Регулярное выражение	Формальный язык для поиска и манипуляций с подстроками в тексте на основе метасимволов.
Инъекция	Атака, при которой вредоносный код вставляется в запросы к базе данных.
Переменная окружения	Значение, используемое в командах и сценариях операционной системы.
Экспоненциальная задержка	Метод повторных попыток с экспоненциальным увеличением времени ожидания.
Аутентификация по схеме Bearer	Схема аутентификации HTTP, использующая токены безопасности (bearer tokens).
Роль пользователя	Категория пользователя с определённым уровнем доступа и правами (USER или ADMIN).
Авторизация	Проверка прав доступа пользователя к определённым ресурсам или действиям.

Продолжение на следующей странице

Термин	**Определение**
GigaChat	Сторонний LLM-провайдер, используемый для генерации ответов на вопросы форм.